Algos condensed and explained

salimek

June 22, 2026

1 Pattern

TITLE

Explanation of the algorithm.

Pseudo-code

```
Algorithm(...)
instruction 1
instruction 2
return result
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Recurrence Formula:

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

2 Algos (everything may not be finished and explained)

INSERTION SORT

Insertion Sort is a simple sorting algorithm that builds the final sorted array one element at a time. At each iteration, the current element is inserted into its correct position inside the already sorted part of the array.

Pseudo-code

```
Insertion-Sort(A, n)

    for j = 2 to n
        key = A[j]
        i = j - 1

        while i > 0 and A[i] > key
            A[i + 1] = A[i]
            i = i - 1

        A[i + 1] = key
```

Time Complexity: $\Theta(n^2)$ worst/average, $\Theta(n)$ best case

Space Complexity: $O(1)$

Recurrence Formula: N/A

Merge Sort

Divide the array into two halves, recursively sort each half, and merge the sorted halves. The objective is to sort an array efficiently using the divide-and-conquer strategy.

Pseudo-code

```
MERGE-SORT(A, p, r)

    if p < r
        q = rounddown((p+r)/2)

        MERGE-SORT(A, p, q)
        MERGE-SORT(A, q+1, r)

        MERGE(A, p, q, r)


MERGE(A,p,q,r)

    n1 = q-p+1
    n2 = r-q

    create arrays L[1...n1+1], R[1...n2+1]

    for i = 1 to n1
        L[i] = A[p+i-1]

    for j = 1 to n2
        R[j] = A[q+j]

    L[n1+1] = 8
    R[n2+1] = 8

    i = 1
    j = 1

    for k = p to r
        if L[i] <= R[j]
            A[k] = L[i]
            i = i + 1

        else
            A[k] = R[j]
            j = j + 1
```

Time Complexity: $\Theta(n \log n)$ in all cases.

Space Complexity: $O(n)$ auxiliary space.

Recurrence Formula: $T(n) = 2T(n/2) + \Theta(n) \Rightarrow T(n) = \Theta(n \log n)$ by Master Theorem (case 2).

Maximum Subarray Optimized

Find the continuous subarray with the maximum possible sum. The algorithm uses divide and conquer by splitting the array, solving both halves recursively, and checking the subarray crossing the middle.

Pseudo-code

```
FIND-MAXIMUM-SUBARRAY(A, low, high)

    if high == low
        return (low, high, A[low])

    mid = (low + high) / 2

    left  = FIND-MAXIMUM-SUBARRAY(A, low, mid)
    right = FIND-MAXIMUM-SUBARRAY(A, mid+1, high)

    cross = FIND-MAX-CROSSING(A, low, mid, high)

    if left-sum >= right-sum and left-sum >= cross-sum
        return (left-low, left-high, left-sum)

    elseif right-sum >= left-sum and right-sum >= cross-sum
        return (right-low, right-high, right-sum)

    else
        return (cross-low, cross-high, cross-sum)


FIND-MAX-CROSSING(A, low, mid, high)

    left-sum = -8
    sum = 0

    for i = mid downto low
        sum = sum + A[i]

        if sum > left-sum
            left-sum = sum
            max-left = i


    right-sum = -8
    sum = 0

    for j = mid+1 to high
        sum = sum + A[j]

        if sum > right-sum
            right-sum = sum
            max-right = j


    return (max-left, max-right, left-sum + right-sum)
```

Time Complexity: $T(n) = 2T(n/2)+\Theta(n) \Rightarrow \Theta(n \log n)$ by Master Theorem.

Space Complexity: $O(\log n)$ auxiliary space due to recursion stack.

Recurrence Formula: $T(n) = 2T(n/2) + \Theta(n)$

Maximum Subarray Brute Force

Checks every possible continuous subarray and keeps the one with the largest sum. The algorithm uses an exhaustive search approach.

Pseudo-code

```
MAXIMUM_SUBARRAY_SLOW(A[1..n])

    B.val = 8
    B.i = 1
    B.j = n

    for i = 1 to n

        tmp = 0

        for j = i to n

            tmp = tmp + A[j]

            if tmp > B.val
                B.val = tmp
                B.i = i
                B.j = j

    return (B.i, B.j, B.val)
```

Time Complexity: $\Theta(n^2)$

Space Complexity: $\Theta(n)$ input space + $\Theta(1)$ auxiliary space.

Recurrence Formula: No recurrence formula (iterative brute force algorithm).

Matrix Multiplication Naive

Multiply two square matrices using the standard triple nested loop approach. Each element of the result matrix is computed by the dot product of a row and a column.

Pseudo-code

```
SQUARE_MAT_MULT(A, B, n)
    create matrix C of size n x n
    for i = 1 to n
        for j = 1 to n
            C[i][j] = 0
            for k = 1 to n
                C[i][j] = C[i][j] + A[i][k] * B[k][j]
    return C
```

Time Complexity: $O(n^3)$

Space Complexity: $O(n^2)$ for the result matrix.

Recurrence Formula: $T(n) = 8T(n/2) + \Theta(n^2) \Rightarrow T(n) = \Theta(n^3)$ by Master Theorem (case 1).

Recursive Matrix Multiplication

Multiply two square matrices using a divide-and-conquer approach. The matrices are divided into smaller submatrices and recursively multiplied.

Pseudo-code

```
REC_MAT_MULT(A, B, n)

    create matrix C of size n x n

    if n == 1

        C11 = A11 * B11


    else

        partition A, B, C into n/2 x n/2 blocks


        C11 = REC_MAT_MULT(A11,B11,n/2)
              + REC_MAT_MULT(A12,B21,n/2)

        C12 = REC_MAT_MULT(A11,B12,n/2)
              + REC_MAT_MULT(A12,B22,n/2)

        C21 = REC_MAT_MULT(A21,B11,n/2)
              + REC_MAT_MULT(A22,B21,n/2)

        C22 = REC_MAT_MULT(A21,B12,n/2)
              + REC_MAT_MULT(A22,B22,n/2)


    return C
```

Time Complexity: $T(n) = 8T(n/2) + \Theta(n^2) \Rightarrow O(n^3)$ by Master Theorem.

Space Complexity: $O(n^2)$ for storing matrices.

Recurrence Formula: $T(n) = 8T(n/2) + \Theta(n^2)$

Strassen's Algorithm

Multiply two matrices using divide and conquer while reducing the number of recursive multiplications from 8 to 7. The algorithm uses linear combinations of submatrices to improve the complexity.

Pseudo-code

```
STRASSEN-MAT-MULT(A, B, n)
    create matrix C of size n x n
    if n == 1
        C11 = A11 * B11

    else
        partition A, B, C into n/2 x n/2 blocks

        M1 = (A11 + A22) * (B11 + B22)
        M2 = (A21 + A22) * B11
        M3 = A11 * (B12 - B22)
        M4 = A22 * (B21 - B11)
        M5 = (A11 + A12) * B22
        M6 = (A21 - A11) * (B11 + B12)
        M7 = (A12 - A22) * (B21 + B22)

        C11 = M1 + M4 - M5 + M7
        C12 = M3 + M5
        C21 = M2 + M4
        C22 = M1 - M2 + M3 + M6

    return C
```

Time Complexity: $T(n) = 7T(n/2) + \Theta(n^2) \Rightarrow \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$.

Space Complexity: $O(n^2)$ for storing matrices.

Recurrence Formula: $T(n) = 7T(n/2) + \Theta(n^2)$

Karatsuba Multiplication

Multiply two large integers faster than the classical multiplication method. The algorithm reduces the number of recursive multiplications from 4 to 3 using a divide-and-conquer strategy.

Pseudo-code

```
KARATSUBA_MULT(x, y, n)
    if n == 1
        return x * y

    else
        split x into xH, xL
        split y into yH, yL

        p1 = KARATSUBA_MULT(xH, yH, n/2)
        p2 = KARATSUBA_MULT(xL, yL, n/2)
        p3 = KARATSUBA_MULT(
             xH + xL,
             yH + yL,
             n/2)

        return p1*b^n
               + (p3 - p1 - p2)*b^(n/2)
               + p2
```

Time Complexity: $T(n) = 3T(n/2) + \Theta(n) \Rightarrow \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$.

Space Complexity: $O(1)$ auxiliary space.

Recurrence Formula: $T(n) = 3T(n/2) + \Theta(n)$

MAX-HEAPIFY

Restore the max-heap property at a given node by comparing it with its children. If a child is larger, swap values and continue recursively down the heap.

Pseudo-code

```
MAX-HEAPIFY(A, i, n)
    l = LEFT(i)
    r = RIGHT(i)

    if l <= n and A[l] > A[i]
        largest = l
    else
        largest = i

    if r <= n and A[r] > A[largest]
        largest = r

    if largest != i
        exchange A[i] with A[largest]
        MAX-HEAPIFY(A, largest, n)
```

Time Complexity: $\Theta(height(i)) = O(\log n)$

Space Complexity: $O(\log n)$ due to recursion stack.

Recurrence Formula: $T(n) = T(n/2) + \Theta(1) \Rightarrow T(n) = O(\log n)$

Build Heap

Transform an unordered array into a valid max-heap. The algorithm applies MAX-HEAPIFY bottom-up on all internal nodes to restore the heap property efficiently.

Pseudo-code

```
BUILD_MAX_HEAP(A, n)
    for i = n/2 downto 1
        MAX_HEAPIFY(A, i, n)
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$ auxiliary space.

Recurrence Formula: No recurrence formula (bottom-up iterative algorithm).

Heapsort

Sort an array using a max-heap structure. The algorithm first builds a max-heap, then repeatedly extracts the maximum element and restores the heap property.

Pseudo-code

```
HEAPSORT(A, n)
    BUILD_MAX_HEAP(A, n)

    for i = n downto 2
        exchange A[1] with A[i]
        MAX_HEAPIFY(A, 1, i-1)
```

Time Complexity: $O(n \log n)$

Space Complexity: $O(1)$ auxiliary space.

Recurrence Formula: No recurrence formula (iterative heap-based algorithm).

Heap Extract Max

Remove and return the maximum element of a max-heap. The root is replaced by the last element, then MAX-HEAPIFY is used to restore the heap property.

Pseudo-code

```
HEAP_EXTRACT_MAX(A, n)
    if n < 1
        error "heap underflow"

    max = A[1]
    A[1] = A[n]
    n = n - 1

    MAX_HEAPIFY(A, 1, n)

    return max
```

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$ auxiliary space.

Recurrence Formula: $T(n) = T(n/2) + \Theta(1) \Rightarrow O(\log n)$

Heap Maximum

Return the maximum element stored in a max-heap. Since the heap property guarantees that the largest element is always at the root, the operation simply accesses the first element of the array.

Pseudo-code

```
HEAP–MAXIMUM(A)
    return A[1]
```

Time Complexity: $O(1)$

Space Complexity: $O(1)$

Recurrence Formula: No recurrence formula (constant-time operation).

Heap Increase Key

Update the value of a node in a max-heap and restore the heap property. After increasing the key, the element is repeatedly swapped with its parent until it reaches a valid position.

Pseudo-code

```
HEAP-INCREASE-KEY(A, i, key)
    if key < A[i]
        error "new key is smaller than current key"
    A[i] = key
    while i > 1 and A[PARENT(i)] < A[i]
        exchange A[i] and A[PARENT(i)]
        i = PARENT(i)
```

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

Recurrence Formula: No recurrence formula (iterative upward traversal).

Max Heap Insert

Insert a new element into a max-heap while preserving the heap property. A temporary value of 8 is first inserted, then HEAP-INCREASE-KEY moves the new key upward until it reaches its correct position.

Pseudo-code

```
MAX-HEAP-INSERT(A, key, n)
    n = n + 1
    A[n] = 8
    HEAP-INCREASE-KEY(A, n, key)
```

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

Recurrence Formula: No recurrence formula (uses HEAP-INCREASE-KEY).

Stack Empty

Check whether a stack contains any elements. The operation simply tests the value of the top pointer.

Pseudo-code

```
STACK_EMPTY(S)
    if S.top == 0
        return TRUE
    else
        return FALSE
```

Time Complexity: $O(1)$

Space Complexity: $O(1)$

Recurrence Formula: No recurrence formula (constant-time operation).

Stack Push

Insert a new element at the top of the stack. The operation increments the top pointer and stores the value.

Pseudo-code

```
PUSH(S, x)
    S.top = S.top + 1
    S[S.top] = x
```

Time Complexity: $O(1)$

Space Complexity: $O(1)$

Recurrence Formula: No recurrence formula (constant-time operation).

Stack Pop

Remove and return the top element of the stack. The operation checks if the stack is empty, then decreases the top pointer.

Pseudo-code

```
POP(S)
    if STACK-EMPTY(S)
        error "underflow"
    else
        S.top = S.top - 1
        return S[S.top + 1]
```

Time Complexity: $O(1)$

Space Complexity: $O(1)$

Recurrence Formula: No recurrence formula (constant-time operation).

Enqueue

Insert a new element at the rear of a queue. The element is stored at the tail position, then the tail pointer is updated using circular array indexing.

Pseudo-code

```
ENQUEUE(Q, x)
    Q[Q.tail] = x

    if Q.tail == Q.length
        Q.tail = 1
    else
        Q.tail = Q.tail + 1
```

Time Complexity: $O(1)$

Space Complexity: $O(1)$

Recurrence Formula: No recurrence formula (constant-time operation).

Dequeue

Remove and return the element at the front of a queue. The element at the head is read, then the head pointer is advanced using circular array indexing.

Pseudo-code

```
DEQUEUE(Q)
    x = Q[Q.head]

    if Q.head == Q.length
        Q.head = 1
    else
        Q.head = Q.head + 1

    return x
```

Time Complexity: $O(1)$

Space Complexity: $O(1)$

Recurrence Formula: No recurrence formula (constant-time operation).

Linked List Search

Search for an element with a given key in a linked list. The algorithm starts from the head and traverses each node until the key is found or the end of the list is reached.

Pseudo-code

```
LIST-SEARCH(L, k)
    x = L.head

    while x != NIL and x.key != k
        x = x.next

    return x
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Recurrence Formula: No recurrence formula (iterative traversal).

Linked List Insert

Insert a new node at the beginning of a doubly linked list. The operation updates the pointers so that the new node becomes the new head while preserving the list connections.

Pseudo-code

```
LIST-INSERT(L, x)
    x.next = L.head

    if L.head != NIL
        L.head.prev = x

    L.head = x
    x.prev = NIL
```

Time Complexity: $O(1)$

Space Complexity: $O(1)$

Recurrence Formula: No recurrence formula (constant-time pointer update).

Linked List Delete

Remove a node from a doubly linked list. The algorithm updates the neighboring nodes' pointers so the list remains correctly connected after the deletion.

Pseudo-code

```
LIST-DELETE(L, x)
    if x.prev != NIL
        x.prev.next = x.next
    else
        L.head = x.next

    if x.next != NIL
        x.next.prev = x.prev
```

Time Complexity: $O(1)$

Space Complexity: $O(1)$

Recurrence Formula: No recurrence formula (constant-time pointer update).

Tree Search

Search for a node with a given key in a binary search tree. The algorithm follows the BST property: go left if the key is smaller, otherwise go right, until the key is found or the tree ends.

Pseudo-code

```
TREE-SEARCH(x, k)
    if x == NIL or k == x.key
        return x

    if k < x.key
        return TREE-SEARCH(x.left, k)
    else
        return TREE-SEARCH(x.right, k)
```

Time Complexity: $O(h)$ where h is the tree height.

Space Complexity: $O(h)$ due to recursive call stack.

Recurrence Formula: $T(h) = T(h-1) + \Theta(1) \Rightarrow O(h)$

Tree Minimum

Find the smallest element in a binary search tree. The minimum value is always located by following left children until reaching a node without a left child.

Pseudo-code

```
TREE-MINIMUM(x)
    while x.left != NIL
        x = x.left

    return x
```

Time Complexity: $O(h)$ where h is the tree height.

Space Complexity: $O(1)$

Recurrence Formula: No recurrence formula (iterative traversal).

Tree Maximum

Find the largest element in a binary search tree. The maximum value is located by following right children until reaching a node without a right child.

Pseudo-code

```
TREE–MAXIMUM(x)
    while x.right != NIL
        x = x.right

    return x
```

Time Complexity: $O(h)$ where h is the tree height.

Space Complexity: $O(1)$

Recurrence Formula: No recurrence formula (iterative traversal).

Tree Successor

Find the next node in the in-order traversal of a binary search tree. If a right subtree exists, the successor is the minimum of that subtree. Otherwise, move upward until finding a node that is a left child of its parent.

Pseudo-code

```
TREE-SUCCESSOR(x)
    if x.right != NIL
        return TREE-MINIMUM(x.right)

    y = x.p

    while y != NIL and x == y.right
        x = y
        y = y.p

    return y
```

Time Complexity: $O(h)$ where h is the tree height.

Space Complexity: $O(1)$

Recurrence Formula: No recurrence formula (iterative tree traversal).

Inorder Tree Walk

Traverse all nodes of a binary search tree in sorted order. The algorithm follows the in-order strategy: left subtree, node, then right subtree.

Pseudo-code

```
INORDER_TREE_WALK(x)
    if x != NIL
        INORDER_TREE_WALK(x.left)
        print key[x]
        INORDER_TREE_WALK(x.right)
```

Time Complexity: $O(n)$

Space Complexity: $O(h)$ recursion stack.

Recurrence Formula: $T(n) = T(n_L) + T(n_R) + \Theta(1) \Rightarrow O(n)$

Preorder Tree Walk

Traverse all nodes of a binary tree by visiting the root first, then the left subtree, and finally the right subtree. This allows processing a node before its children.

Pseudo-code

```
PREORDER-TREE-WALK(x)
    if x != NIL
        print key[x]
        PREORDER-TREE-WALK(x.left)
        PREORDER-TREE-WALK(x.right)
```

Time Complexity: $O(n)$

Space Complexity: $O(h)$ recursion stack.

Recurrence Formula: $T(n) = T(n_L) + T(n_R) + \Theta(1) \Rightarrow O(n)$

Postorder Tree Walk

Traverse all nodes of a binary tree by visiting the children first and the root last. The traversal order is: left subtree, right subtree, then node.

Pseudo-code

```
POSTORDER_TREE_WALK(x)
    if x != NIL
        POSTORDER_TREE_WALK(x.left)
        POSTORDER_TREE_WALK(x.right)
        print key[x]
```

Time Complexity: $O(n)$

Space Complexity: $O(h)$ recursion stack.

Recurrence Formula: $T(n) = T(n_L) + T(n_R) + \Theta(1) \Rightarrow O(n)$

Tree Insert

Insert a new node into a binary search tree while preserving the BST property. The algorithm searches for the correct position and attaches the node as a leaf.

Pseudo-code

```
TREE-INSERT(T, z)
    y = NIL
    x = T.root

    while x != NIL
        y = x
        if z.key < x.key
            x = x.left
        else
            x = x.right

    z.p = y

    if y == NIL
        T.root = z
    elseif z.key < y.key
        y.left = z
    else
        y.right = z
```

Time Complexity: $O(h)$ where h is the tree height.

Space Complexity: $O(1)$

Recurrence Formula: No recurrence formula (iterative tree traversal).

Tree Delete

Remove a node from a binary search tree while preserving the BST property. The algorithm handles the three cases: no child, one child, or two children. It uses TRANSPLANT to safely reconnect subtrees.

Pseudo-code

```
TRANSPLANT(T, u, v)
    if u.p == NIL
        T.root = v
    elseif u == u.p.left
        u.p.left = v
    else
        u.p.right = v

    if v != NIL
        v.p = u.p


TREE-DELETE(T, z)
    if z.left == NIL
        TRANSPLANT(T, z, z.right)

    elseif z.right == NIL
        TRANSPLANT(T, z, z.left)

    else
        y = TREE-MINIMUM(z.right)

        if y.p != z
            TRANSPLANT(T, y, y.right)
            y.right = z.right
            y.right.p = y

        TRANSPLANT(T, z, y)
        y.left = z.left
        y.left.p = y
```

Time Complexity: $O(h)$ where h is the tree height.

Space Complexity: $O(1)$

Recurrence Formula: No recurrence formula (iterative tree modification).

Rod Cutting Algorithm

Find the maximum revenue obtainable by cutting a rod of length n. The dynamic programming approach uses memoization to avoid recomputing the same subproblems.

Pseudo-code

```
Memoized-Cut-Rod(p, n)
    let r[0..n] be a new array
    for i = 0 to n
        r[i] = 8

    return Memoized-Cut-Rod-Aux(p, n, r)
```

Time Complexity: $O(n^2)$

Space Complexity: $O(n)$

Recurrence Formula: $T(n) = \sum_{j=1}^{n} T(n-j) + O(n) \Rightarrow O(n^2)$ with memoization.

Memoized Cut Rod Aux

Compute the optimal revenue for a rod of length n using memoization. The algorithm stores previously computed solutions to avoid solving the same subproblems multiple times.

Pseudo-code

```
Memoized-Cut-Rod-Aux(p, n, r)
    if r[n] >= 0
        return r[n]

    if n == 0
        q = 0
    else
        q = 8
        for i = 1 to n
            q = max(q,
                p[i] +
                Memoized-Cut-Rod-Aux(p, n-i, r))

    r[n] = q
    return q
```

Time Complexity: $O(n^2)$

Space Complexity: $O(n)$

Recurrence Formula: $T(n) = \sum_{i=1}^{n} T(n-i) + O(n)$ with memoization $\Rightarrow O(n^2)$.

Bottom-Up Cut Rod

Compute the maximum revenue obtainable from a rod of length n using bottom-up dynamic programming. The algorithm solves smaller subproblems first and stores their results to build the final optimal solution.

Pseudo-code

```
BOTTOM-UP-CUT-ROD(p, n)
    let r[0..n] be a new array
    r[0] = 0

    for j = 1 to n
        q = 8
        for i = 1 to j
            q = max(q,
                    p[i] + r[j-i])

        r[j] = q

    return r[n]
```

Time Complexity: $O(n^2)$

Space Complexity: $O(n)$

Recurrence Formula: $T(n) = \sum_{i=1}^{n} T(n-i) + \Theta(n) \Rightarrow O(n^2)$ with dynamic programming.

Matrix Chain Order

Find the optimal parenthesization of a matrix chain to minimize the number of scalar multiplications. The algorithm uses dynamic programming by storing optimal costs for smaller subchains.

Pseudo-code

```
MATRIX-CHAIN-ORDER(p)
    n = p.length - 1
    let m[1..n,1..n] and s[1..n,1..n]

    for i = 1 to n
        m[i,i] = 0

    for l = 2 to n
        for i = 1 to n-l+1
            j = i+l-1
            m[i,j] = 8

            for k = i to j-1
                q = m[i,k]
                    + m[k+1,j]
                    + p[i-1]*p[k]*p[j]

                if q < m[i,j]
                    m[i,j] = q
                    s[i,j] = k

    return m and s
```

Time Complexity: $O(n^3)$

Space Complexity: $O(n^2)$

Recurrence Formula: $m[i,j] = \min_{i \leq k < j}(m[i,k] + m[k+1,j] + p[i-1]p[k]p[j])$

Longest Common Subsequence

Find the longest subsequence appearing in the same order in two sequences. The algorithm uses dynamic programming by storing solutions of smaller prefixes to build the optimal subsequence.

Pseudo-code

```
LCS-LENGTH(X, Y, m, n)
    let b[1..m,1..n] and c[0..m,0..n]

    for i = 1 to m
        c[i,0] = 0

    for j = 0 to n
        c[0,j] = 0

    for i = 1 to m
        for j = 1 to n
            if x_i == y_j
                c[i,j] = c[i-1,j-1] + 1
                b[i,j] = "<-"

            else if c[i-1,j] >= c[i,j-1]
                c[i,j] = c[i-1,j]
                b[i,j] = "up"

            else
                c[i,j] = c[i,j-1]
                b[i,j] = "<-"

    return c and b
```

Time Complexity: $O(mn)$

Space Complexity: $O(mn)$

Recurrence Formula:
If $x_i = y_j$: $c[i,j] = c[i-1,j-1] + 1$
Else: $c[i,j] = \max(c[i-1,j], c[i,j-1])$

Print LCS

Reconstruct the longest common subsequence using the direction table computed by the LCS dynamic programming algorithm. The algorithm follows the stored arrows to rebuild the optimal sequence.

Pseudo-code

```
PRINT–LCS(b, X, i, j)
    if i == 0 or j == 0
        return

    if b[i,j] == "diagonal"
        PRINT–LCS(b, X, i−1, j−1)
        print x_i

    elseif b[i,j] == "up"
        PRINT–LCS(b, X, i−1, j)

    else
        PRINT–LCS(b, X, i, j−1)
```

Time Complexity: $O(m+n)$

Space Complexity: $O(m+n)$ recursion stack.

Recurrence Formula:
$T(i,j) = T(i-1,j-1)+\Theta(1)$ or $T(i,j) = T(i-1,j)+\Theta(1)$ depending on direction.

Bottom-Up Optimal BST

Construct a binary search tree with minimum expected search cost. The algorithm uses dynamic programming to find the optimal root and recursively builds the best possible subtrees.

Pseudo-code

```
OPTIMAL-BST(p, q, n)
    let e, w, root be new tables

    for i = 1 to n+1
        e[i,i-1] = 0
        w[i,i-1] = 0

    for l = 1 to n
        for i = 1 to n-l+1
            j = i+l-1
            e[i,j] = 8
            w[i,j] = w[i,j-1] + p[j]

            for r = i to j
                t = e[i,r-1]
                    + e[r+1,j]
                    + w[i,j]

                if t < e[i,j]
                    e[i,j] = t
                    root[i,j] = r

    return e and root
```

Time Complexity: $O(n^3)$

Space Complexity: $O(n^2)$

Recurrence Formula:
$e[i,j] = \min_{i \leq r \leq j}(e[i,r-1] + e[r+1,j] + w[i,j])$

Breadth-First Search

Explore a graph level by level starting from a source vertex s. BFS computes the shortest distance in number of edges from s to every reachable vertex by using a queue.

Pseudo-code

```
BFS(G, s)
    for each u in V - {s}
        u.d = 8

    s.d = 0
    Q = empty queue
    ENQUEUE(Q, s)

    while Q != empty
        u = DEQUEUE(Q)

        for each v in Adj[u]
            if v.d == 8
                v.d = u.d + 1
                ENQUEUE(Q, v)
```

Time Complexity: $O(V + E)$

Space Complexity: $O(V)$

Recurrence Formula:
$d[v] = d[u] + 1$ when v is discovered from u.

Depth-First Search

Explore a graph by going as deep as possible along each branch before backtracking. DFS assigns discovery and finishing times to vertices and uses colors to track the exploration state.

Pseudo-code

```
DFS(G)
    for each u in G.V
        u.color = WHITE

    time = 0

    for each u in G.V
        if u.color == WHITE
            DFS-VISIT(G,u)

DFS-VISIT(G, u)
    time = time + 1
    u.d = time
    u.color = GRAY

    for each v in Adj[u]
        if v.color == WHITE
            DFS-VISIT(G,v)

    u.color = BLACK
    time = time + 1
    u.f = time
```

Time Complexity: $O(V + E)$

Space Complexity: $O(V)$

Recurrence Formula:
$T(V, E) = T(V - 1, E) + O(E) \Rightarrow O(V + E)$

Topological Sort

Produce a linear ordering of the vertices of a Directed Acyclic Graph (DAG) such that for every edge (u, v), u appears before v. The algorithm uses DFS finishing times to determine the ordering.

Pseudo-code

```
TOPOLOGICAL-SORT(G)
    DFS(G)
    // compute finishing times v.f

    output vertices in decreasing order
    of their finishing times
```

Time Complexity: $O(V + E)$

Space Complexity: $O(V)$

Recurrence Formula:
DFS traversal: $T(V, E) = O(V + E)$

Strongly Connected Components

Decompose a directed graph into maximal sets of vertices where every vertex is reachable from every other vertex. The algorithm uses two DFS passes and the transpose graph.

Pseudo-code

```
SCC(G)
    DFS(G)
    // compute finishing times u.f

    Compute G^T
    // reverse all edges

    DFS(G^T)
    considering vertices
    in decreasing order of u.f

    output each DFS tree
    as one SCC
```

Time Complexity: $O(V + E)$

Space Complexity: $O(V + E)$

Recurrence Formula:
Two DFS traversals: $T(V, E) = 2O(V + E) = O(V + E)$

Maximum Flow - Ford Fulkerson

Find the maximum possible flow from a source s to a sink t in a flow network while respecting capacities. The algorithm repeatedly finds augmenting paths in the residual graph and increases the flow until no improvement is possible.

Pseudo-code

```
FORD–FULKERSON(G, s, t)
    initialize flow f = 0

    while there exists an
    augmenting path p in Gf

        augment flow f along p

        update residual graph

    return f
```

Time Complexity:
$O(E \cdot |f^*|)$
for integer capacities.

Space Complexity:
$O(V + E)$

Recurrence Formula:
$f = f + f_p$
where f_p is the bottleneck capacity of the augmenting path.

Connected Components

Find all connected components of an undirected graph using the Union-Find data structure. Each vertex starts in its own set, and edges merge sets that belong to the same component.

Pseudo-code

```
CONNECTED-COMPONENTS(G)
    for each vertex v in G.V
        MAKE-SET(v)

    for each edge (u,v) in G.E
        if FIND-SET(u) != FIND-SET(v)
            UNION(u,v)
```

Time Complexity:
$O(E \cdot \alpha(V))$

Space Complexity:
$O(V)$

Recurrence Formula:
Union-Find operations: $O(\alpha(V))$ amortized per operation.

MAKE-SET

Create a new disjoint set containing only one element. Each element starts as its own root in the Union-Find structure.

Pseudo-code

```
MAKE-SET(x)
    x.p = x
    x.rank = 0
```

Time Complexity:
$O(1)$

Space Complexity:
$O(1)$ per element

Recurrence Formula:
Single-node tree initialization.

FIND-SET

Find the representative (root) of the set containing an element. The operation uses path compression to flatten the tree and make future queries faster.

Pseudo-code

```
FIND–SET(x)
    if x != x.p
        x.p = FIND–SET(x.p)

    return x.p
```

Time Complexity:
$O(\alpha(n))$ amortized

Space Complexity:
$O(1)$ auxiliary

Recurrence Formula:
Path compression: $T(n) \approx O(\alpha(n))$

UNION

Merge two disjoint sets into one set. The operation finds the representatives of both sets and links the two trees together using the Union-Find structure.

Pseudo-code

```
UNION(x, y)

    LINK(FIND–SET(x),
         FIND–SET(y))
```

Time Complexity:
$O(\alpha(n))$ amortized

Space Complexity:
$O(1)$ auxiliary

Recurrence Formula:
Union by rank: $T(n) \approx O(\alpha(n))$

LINK

Attach one root under another in the Union-Find structure. The operation uses union by rank to keep the trees shallow and maintain an efficient balanced forest.

Pseudo-code

```
LINK(x, y)
    if x.rank > y.rank
        y.p = x
    else
        x.p = y
        if x.rank == y.rank
            y.rank = y.rank + 1
```

Time Complexity:
$O(1)$

Space Complexity:
$O(1)$ auxiliary

Recurrence Formula:
Union by rank: keeps tree height bounded by $O(\log n)$.

PRIM'S ALGORITHM

Prim's algorithm builds a Minimum Spanning Tree (MST) of a weighted graph. It repeatedly chooses the minimum weight edge connecting the current tree to a new vertex using a priority queue.

Pseudo-code

```
PRIM(G, w, r)

    Q = empty priority queue

    for each u in G.V
        u.key = 8
        u.pi = NIL
        INSERT(Q, u)

    DECREASE_KEY(Q, r, 0)

    while Q != empty

        u = EXTRACT_MIN(Q)

        for each v in G.Adj[u]

            if v in Q and w(u,v) < v.key

                v.pi = u
                DECREASE_KEY(Q, v, w(u,v))
```

Time Complexity:
$O(E \log V)$ with binary heap
$O(E + V \log V)$ with Fibonacci heap

Space Complexity:
$O(V)$

Recurrence Formula:
Greedy expansion:
$T(V, E) = O(E \log V)$

KRUSKAL'S ALGORITHM

Kruskal's algorithm constructs a Minimum Spanning Tree (MST) by selecting the smallest weight edges while avoiding cycles using the Union-Find data structure.

Pseudo-code

```
KRUSKAL(G, w)
    A = empty set
    for each vertex v in G.V
        MAKE-SET(v)
    sort edges of G.E by increasing weight
    for each edge (u,v) in sorted order
        if FIND-SET(u) != FIND-SET(v)
            A = A U {(u,v)}
            UNION(u,v)
    return A
```

Time Complexity:
$O(E \log E)$
$= O(E \log V)$

Space Complexity:
$O(V)$

Recurrence Formula:
Sorting dominates:
$T(E) = O(E \log E)$

RELAX

Relaxation improves the shortest path estimate of a vertex by checking whether passing through another vertex gives a shorter route.

Pseudo-code

```
RELAX(u, v, w)
    if v.d > u.d + w(u,v)
        v.d = u.d + w(u,v)
        v.pi = u
```

Time Complexity:
$O(1)$

Space Complexity:
$O(1)$

Recurrence Formula:
Shortest path update:
$d[v] = \min(d[v], d[u] + w(u, v))$

BELLMAN-FORD

Bellman-Ford computes shortest paths from a source vertex and supports negative edge weights. It repeatedly relaxes all edges to improve estimates.

Pseudo-code

```
BELLMAN-FORD(G, w, s)
    INIT-SINGLE-SOURCE(G, s)
    for i = 1 to |G.V|-1
        for each edge (u,v) in G.E
            RELAX(u,v,w)

    return TRUE
```

Time Complexity:
$O(VE)$

Space Complexity:
$O(V)$

Recurrence Formula:
$V - 1$ relaxations:
$T(V, E) = O(VE)$

DIJKSTRA

Dijkstra's algorithm computes shortest paths from a source vertex in a graph with non-negative edge weights. It repeatedly selects the closest unprocessed vertex and relaxes its outgoing edges.

Pseudo-code

```
DIJKSTRA(G,w,s)
    INIT-SINGLE-SOURCE(G,s)
    S = empty set
    Q = G.V
    while Q != empty
        u = EXTRACT-MIN(Q)
        S = S U {u}

        for each v in G.Adj[u]
            RELAX(u,v,w)
```

Time Complexity:
$O(E \log V)$ with binary heap
$O(V \log V + E)$ with efficient decrease-key

Space Complexity:
$O(V)$

Recurrence Formula:
Greedy shortest path:
$T(V, E) = O((V + E) \log V)$